03 — So sánh GAN 2D: Normalized vs Unnormalized

Dự án: Latent Manipulation of Brain MRI using Volume-Preserving GANs

Mục tiêu: So sánh 2 model từ file 01 và 02, chọn model tốt hơn để dùng cho GAN 3D.

Metrics:

- **Loss_G** — loss Generator cuối cùng (thấp hơn = tốt hơn)
- **SSIM** — độ giữ nguyên cấu trúc giải phẫu (cao hơn = tốt hơn)

Output:

```
compare_2d/
└── comparison_result.json   ← kết quả so sánh, model nào tốt hơn
```

Bước 1: Cấu hình

```
import os
# ==== ĐƯỜNG DẪN 2 MODEL ====
MODEL_NORM_PATH   =
'/kaggle/input/datasets/cminhnguyndsdsds/conditional-gan2d-
normalized/best_model.pth'
MODEL_UNNORM_PATH = '/kaggle/input/datasets/dyio147/conditional-gan2d-
unnormalized/best_model.pth'

# ==== DATA ====
DATA_NORM_DIR   = '/kaggle/input/datasets/minhbodoi/full-preprocessed-
2d/preprocessed_2d/normalized'
DATA_UNNORM_DIR = '/kaggle/input/datasets/minhbodoi/full-preprocessed-
2d/preprocessed_2d/unnormalized'
LABELS_CSV      = '/kaggle/input/datasets/minhbodoi/full-preprocessed-
2d/preprocessed_2d/preprocessing_log.csv'

OUTPUT_DIR = '/kaggle/working/compare_2d'
os.makedirs(OUTPUT_DIR, exist_ok=True)

IMAGE_SIZE = 256
LATENT_DIM = 256
print('Câú hình xong!')

Câú hình xong!
```

Bước 2: Import thư viện

```
!pip install scikit-image -q

import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import json
from PIL import Image
from skimage.metrics import structural_similarity as ssim
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
import warnings
warnings.filterwarnings('ignore')

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Device: {DEVICE}')
```

```
Device: cuda
```

Bước 3: Load model

```
class AgeEmbedding(nn.Module):
    def __init__(self, embed_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 128), nn.ReLU(),
            nn.Linear(128, embed_dim)
        )
    def forward(self, age):
        return self.net(age.unsqueeze(-1))


class UNetBlock(nn.Module):
    def __init__(self, in_ch, out_ch, down=True, use_bn=True,
dropout=False):
        super().__init__()
        layers = []
        if down : layers.append(nn.Conv2d(in_ch, out_ch, 4, 2, 1,
bias=False))
        else    : layers.append(nn.ConvTranspose2d(in_ch, out_ch, 4,
2, 1, bias=False))
        if use_bn  : layers.append(nn.BatchNorm2d(out_ch))
        if dropout : layers.append(nn.Dropout(0.5))
        layers.append(nn.LeakyReLU(0.2) if down else nn.ReLU())
        self.block = nn.Sequential(*layers)
    def forward(self, x):
        return self.block(x)
```

```
class Generator(nn.Module):
    def __init__(self, latent_dim=256):
        super().__init__()
        self.age_embed = AgeEmbedding(latent_dim)
        self.age_proj  = nn.Linear(latent_dim, 512)
        self.e1 = UNetBlock(1,   64,  down=True, use_bn=False)
        self.e2 = UNetBlock(64,  128, down=True)
        self.e3 = UNetBlock(128, 256, down=True)
        self.e4 = UNetBlock(256, 512, down=True)
        self.e5 = UNetBlock(512, 512, down=True)
        self.e6 = UNetBlock(512, 512, down=True)
        self.e7 = UNetBlock(512, 512, down=True)
        self.e8 = UNetBlock(512, 512, down=True, use_bn=False)
        self.d1 = UNetBlock(512,  512, down=False, dropout=True)
        self.d2 = UNetBlock(1024, 512, down=False, dropout=True)
        self.d3 = UNetBlock(1024, 512, down=False, dropout=True)
        self.d4 = UNetBlock(1024, 512, down=False)
        self.d5 = UNetBlock(1024, 256, down=False)
        self.d6 = UNetBlock(512,  128, down=False)
        self.d7 = UNetBlock(256,  64,  down=False)
        self.out = nn.Sequential(nn.ConvTranspose2d(128, 1, 4, 2, 1),
nn.Tanh())

    def forward(self, x, age):
        e1 = self.e1(x);  e2 = self.e2(e1); e3 = self.e3(e2); e4 =
self.e4(e3)
        e5 = self.e5(e4); e6 = self.e6(e5); e7 = self.e7(e6); e8 =
self.e8(e7)
        z  = e8 + self.age_proj(self.age_embed(age)).view(-1, 512, 1,
1)
        d1 = self.d1(z)
        d2 = self.d2(torch.cat([d1, e7], dim=1))
        d3 = self.d3(torch.cat([d2, e6], dim=1))
        d4 = self.d4(torch.cat([d3, e5], dim=1))
        d5 = self.d5(torch.cat([d4, e4], dim=1))
        d6 = self.d6(torch.cat([d5, e3], dim=1))
        d7 = self.d7(torch.cat([d6, e2], dim=1))
        return self.out(torch.cat([d7, e1], dim=1))



# ── StyleGAN classes (để load checkpoint mới) ──────────────────
class MappingNetwork(nn.Module):
    def __init__(self, latent_dim=256, w_dim=512, n_layers=4):
        super().__init__()
        layers = [AgeEmbedding(latent_dim), nn.ReLU()]
        in_dim = latent_dim
        for _ in range(n_layers - 1):
```

```
            layers += [nn.Linear(in_dim, w_dim), nn.ReLU()]
            in_dim = w_dim
        self.net = nn.Sequential(*layers)
        self.out = nn.Linear(in_dim, w_dim)
    def forward(self, age): return self.out(self.net(age))

class AdaIN2D(nn.Module):
    def __init__(self, channels, w_dim=512):
        super().__init__()
        self.norm  = nn.InstanceNorm2d(channels, affine=False)
        self.style = nn.Linear(w_dim, channels * 2)
    def forward(self, x, w):
        style = self.style(w).unsqueeze(-1).unsqueeze(-1)
        scale, shift = style.chunk(2, dim=1)
        return scale * self.norm(x) + shift

class Generator2D_StyleGAN(nn.Module):
    def __init__(self, latent_dim=256, w_dim=512):
        super().__init__()
        self.mapping = MappingNetwork(latent_dim, w_dim)
        self.e1=UNetBlock(1,64,down=True,use_bn=False);
self.e2=UNetBlock(64,128,down=True)
        self.e3=UNetBlock(128,256,down=True);
self.e4=UNetBlock(256,512,down=True)
        self.e5=UNetBlock(512,512,down=True);
self.e6=UNetBlock(512,512,down=True)
        self.e7=UNetBlock(512,512,down=True);
self.e8=UNetBlock(512,512,down=True,use_bn=False)

self.d1=nn.Sequential(nn.ConvTranspose2d(512,512,4,2,1,bias=False),nn.
Dropout(0.5),nn.ReLU())

self.d2=nn.Sequential(nn.ConvTranspose2d(1024,512,4,2,1,bias=False),nn
.Dropout(0.5),nn.ReLU())

self.d3=nn.Sequential(nn.ConvTranspose2d(1024,512,4,2,1,bias=False),nn
.Dropout(0.5),nn.ReLU())

self.d4=nn.Sequential(nn.ConvTranspose2d(1024,512,4,2,1,bias=False),nn
.ReLU())

self.d5=nn.Sequential(nn.ConvTranspose2d(1024,256,4,2,1,bias=False),nn
.ReLU())

self.d6=nn.Sequential(nn.ConvTranspose2d(512,128,4,2,1,bias=False),nn.
ReLU())

self.d7=nn.Sequential(nn.ConvTranspose2d(256,64,4,2,1,bias=False),nn.R
eLU())
```

```
self.out=nn.Sequential(nn.ConvTranspose2d(128,1,4,2,1),nn.Tanh())
        self.adain1=AdaIN2D(512,w_dim); self.adain2=AdaIN2D(512,w_dim)
        self.adain3=AdaIN2D(512,w_dim); self.adain4=AdaIN2D(512,w_dim)
        self.adain5=AdaIN2D(256,w_dim); self.adain6=AdaIN2D(128,w_dim)
        self.adain7=AdaIN2D(64,w_dim)
    def forward(self, x, age):
        w=self.mapping(age)
        e1=self.e1(x); e2=self.e2(e1); e3=self.e3(e2); e4=self.e4(e3)
        e5=self.e5(e4); e6=self.e6(e5); e7=self.e7(e6); e8=self.e8(e7)
        d1=self.adain1(self.d1(e8),w);
d2=self.adain2(self.d2(torch.cat([d1,e7],1)),w)
        d3=self.adain3(self.d3(torch.cat([d2,e6],1)),w);
d4=self.adain4(self.d4(torch.cat([d3,e5],1)),w)
        d5=self.adain5(self.d5(torch.cat([d4,e4],1)),w);
d6=self.adain6(self.d6(torch.cat([d5,e3],1)),w)
        d7=self.adain7(self.d7(torch.cat([d6,e2],1)),w)
        return self.out(torch.cat([d7,e1],1))


# ── Kiến trúc concat (age_fuse) ──────────────────────────────
class Generator_Concat(nn.Module):
    """Conditional GAN với concat thay vì + ở bottleneck."""
    def __init__(self, latent_dim=256):
        super().__init__()
        self.age_embed = AgeEmbedding(latent_dim)
        self.age_proj  = nn.Linear(latent_dim, 512)
        self.age_fuse  = nn.Conv2d(1024, 512, 1)
        self.e1 = UNetBlock(1,   64,  down=True, use_bn=False)
        self.e2 = UNetBlock(64,  128, down=True)
        self.e3 = UNetBlock(128, 256, down=True)
        self.e4 = UNetBlock(256, 512, down=True)
        self.e5 = UNetBlock(512, 512, down=True)
        self.e6 = UNetBlock(512, 512, down=True)
        self.e7 = UNetBlock(512, 512, down=True)
        self.e8 = UNetBlock(512, 512, down=True, use_bn=False)
        self.d1 = UNetBlock(512,  512, down=False, dropout=True)
        self.d2 = UNetBlock(1024, 512, down=False, dropout=True)
        self.d3 = UNetBlock(1024, 512, down=False, dropout=True)
        self.d4 = UNetBlock(1024, 512, down=False)
        self.d5 = UNetBlock(1024, 256, down=False)
        self.d6 = UNetBlock(512,  128, down=False)
        self.d7 = UNetBlock(256,  64,  down=False)
        self.out = nn.Sequential(nn.ConvTranspose2d(128, 1, 4, 2, 1),
nn.Tanh())

    def forward(self, x, age):
        e1=self.e1(x); e2=self.e2(e1); e3=self.e3(e2); e4=self.e4(e3)
        e5=self.e5(e4); e6=self.e6(e5); e7=self.e7(e6); e8=self.e8(e7)
        age_feat = self.age_proj(self.age_embed(age)).view(-
```

```
1,512,1,1).expand_as(e8)
        z = self.age_fuse(torch.cat([e8, age_feat], dim=1))
        d1=self.d1(z); d2=self.d2(torch.cat([d1,e7],1))
        d3=self.d3(torch.cat([d2,e6],1));
d4=self.d4(torch.cat([d3,e5],1))
        d5=self.d5(torch.cat([d4,e4],1));
d6=self.d6(torch.cat([d5,e3],1))
        d7=self.d7(torch.cat([d6,e2],1))
        return self.out(torch.cat([d7,e1],1))

def load_generator(ckpt_path):
    ckpt = torch.load(ckpt_path, map_location=DEVICE)
    keys = list(ckpt['G_state'].keys())
    if any('mapping' in k for k in keys):
        G = Generator2D_StyleGAN(LATENT_DIM).to(DEVICE)
        print('  -> StyleGAN architecture detected')
    elif any('age_fuse' in k for k in keys):
        G = Generator_Concat(LATENT_DIM).to(DEVICE)
        print('  -> Conditional GAN Concat architecture detected')
    else:
        G = Generator(LATENT_DIM).to(DEVICE)
        print('  -> U-Net architecture detected')
    G.load_state_dict(ckpt['G_state'])
    G.eval()
    return G, ckpt


print('Loading Normalized...')
G_norm,   ckpt_norm   = load_generator(MODEL_NORM_PATH)
print('Loading Unnormalized...')
G_unnorm, ckpt_unnorm = load_generator(MODEL_UNNORM_PATH)

print(f'Normalized   — epoch: {ckpt_norm["epoch"]} | best_val_SSIM:
{ckpt_norm.get("best_val_ssim",-1):.4f}')
print(f'Unnormalized — epoch: {ckpt_unnorm["epoch"]} | best_val_SSIM:
{ckpt_unnorm.get("best_val_ssim",-1):.4f}')
```

```
Loading Normalized...
  -> Conditional GAN Concat architecture detected
Loading Unnormalized...
  -> Conditional GAN Concat architecture detected
Normalized   — epoch: 44 | best_val_SSIM: 0.9778
Unnormalized — epoch: 22 | best_val_SSIM: 0.9751
```

Bước 4: Dataset + DataLoader

```
class BrainMRI2DDataset(Dataset):
    def __init__(self, data_dir, labels_csv, age_min, age_max,
image_size=256):
```

```
        self.age_min = age_min
        self.age_max = age_max
        df = pd.read_csv(labels_csv)
        df['full_path'] = df['png_file'].apply(lambda x:
os.path.join(data_dir, x))
        self.df =
df[df['full_path'].apply(os.path.exists)].reset_index(drop=True)
        self.transform = transforms.Compose([
            transforms.Resize((image_size, image_size)),
            transforms.ToTensor(),
            transforms.Normalize([0.5], [0.5])
        ])

    def normalize_age(self, age):
        return 2 * (age - self.age_min) / (self.age_max -
self.age_min) - 1

    def __len__(self): return len(self.df)

    def __getitem__(self, idx):
        row      = self.df.iloc[idx]
        img      =
self.transform(Image.open(row['full_path']).convert('L'))
        age_norm = torch.tensor(self.normalize_age(row['age']),
dtype=torch.float32)
        return img, age_norm


loader_norm   = DataLoader(
    BrainMRI2DDataset(DATA_NORM_DIR,   LABELS_CSV,
ckpt_norm['age_min'],   ckpt_norm['age_max']),
    batch_size=8, shuffle=False
)
loader_unnorm = DataLoader(
    BrainMRI2DDataset(DATA_UNNORM_DIR, LABELS_CSV,
ckpt_unnorm['age_min'], ckpt_unnorm['age_max']),
    batch_size=8, shuffle=False
)
print('Dataset săñ sàng!')
```

```
Dataset săñ sàng!
```

Bước 5: Tính SSIM và so sánh

```
def compute_ssim(G, loader, device):
    """Tính SSIM trung bình giữa aʾnh real và aʾnh fake."""
    scores = []
    G.eval()
    with torch.no_grad():
        for real_imgs, ages_norm in loader:
```

```
            real_imgs = real_imgs.to(device)
            ages_norm = ages_norm.to(device)
            fake_imgs = G(real_imgs, ages_norm)
            for i in range(real_imgs.size(0)):
                real_np = (real_imgs[i, 0].cpu().numpy() + 1) / 2
                fake_np = (fake_imgs[i, 0].cpu().numpy() + 1) / 2
                scores.append(ssim(real_np, fake_np, data_range=1.0))
    return float(np.mean(scores))


print('Đang tính SSIM...')
ssim_norm   = compute_ssim(G_norm,   loader_norm,   DEVICE)
ssim_unnorm = compute_ssim(G_unnorm, loader_unnorm, DEVICE)

# Chấm điểm
score_norm = score_unnorm = 0
if ckpt_norm.get('best_val_ssim',0) >
ckpt_unnorm.get('best_val_ssim',0): score_norm   += 1
else                                                    :
score_unnorm += 1
if ssim_norm > ssim_unnorm                              :
score_norm   += 1
else                                                    :
score_unnorm += 1

winner = 'normalized' if score_norm >= score_unnorm else
'unnormalized'

# In kết quả
print(f'\n{"Metric":<20} {"Normalized":>12} {"Unnormalized":>14}')
print('-' * 48)
print(f'{"best_val_SSIM":<20} {ckpt_norm.get("best_val_ssim",-
1):>12.4f} {ckpt_unnorm.get("best_val_ssim",-1):>14.4f}')
print(f'{"SSIM":<20} {ssim_norm:>12.4f} {ssim_unnorm:>14.4f}')
print(f'{"Score":<20} {score_norm:>12}/2 {score_unnorm:>13}/2')
print(f'\n→ Model tốt hơn: {winner.upper()}')
print('→ Dùng model này cho GAN 3D!')

# Lưu kết quả
result = {
    'winner'       : winner,
    'normalized'   : {'best_val_ssim': ckpt_norm.get('best_val_ssim',-
1),   'ssim': ssim_norm},
    'unnormalized' : {'best_val_ssim':
ckpt_unnorm.get('best_val_ssim',-1), 'ssim': ssim_unnorm},
}
with open(f'{OUTPUT_DIR}/comparison_result.json', 'w') as f:
    json.dump(result, f, indent=2)
print(f'\nKết quả lưu tại: {OUTPUT_DIR}/comparison_result.json')
```

```
Đang tính SSIM...

Metric                  Normalized   Unnormalized
------------------------------------------------
best_val_SSIM               0.9778         0.9751
SSIM                        0.9800         0.9749
Score                            2/2             0/2

→ Model tốt hơn: NORMALIZED
→ Dùng model này cho GAN 3D!

Kết quả lưu tại: /kaggle/working/compare_2d/comparison_result.json
```

```python
# Tự động lưu kết quả thành Kaggle Dataset
import json, os, subprocess

dataset_name = os.path.basename(OUTPUT_DIR).replace('_', '-')
kaggle_user  = [l.split(':')[1].strip() for l in
                subprocess.run('kaggle config view', shell=True,
                               capture_output=True, text=True)
                .stdout.split('\n') if 'username' in l][0]

with open(f'{OUTPUT_DIR}/dataset-metadata.json', 'w') as f:
    json.dump({
        'title'   : dataset_name,
        'id'      : f'{kaggle_user}/{dataset_name}',
        'licenses': [{'name': 'CC0-1.0'}]
    }, f)

check = subprocess.run(f'kaggle datasets list --user {kaggle_user} --
search {dataset_name}',
                       shell=True, capture_output=True, text=True)
if dataset_name in check.stdout:
    os.system(f'kaggle datasets version -p {OUTPUT_DIR} -m "update"')
else:
    os.system(f'kaggle datasets create -p {OUTPUT_DIR}')

print(f'Done! {kaggle_user}/{dataset_name}')
```

```
Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.1), please consider upgrading to the latest version
(2.0.2)
Starting upload for file comparison_result.json

100%|██████████| 219/219 [00:00<00:00, 751B/s]

Upload successful: comparison_result.json (219B)
Dataset version is being created. Please check progress at
https://www.kaggle.com/datasets/minhbodoi/compare-2d
Done! minhbodoi/compare-2d
```